

### Tarea 3: Algoritmos de Ordenamiento en ARM64

**Estudiante:** Carlos Alfonzo Jared González Sagastume

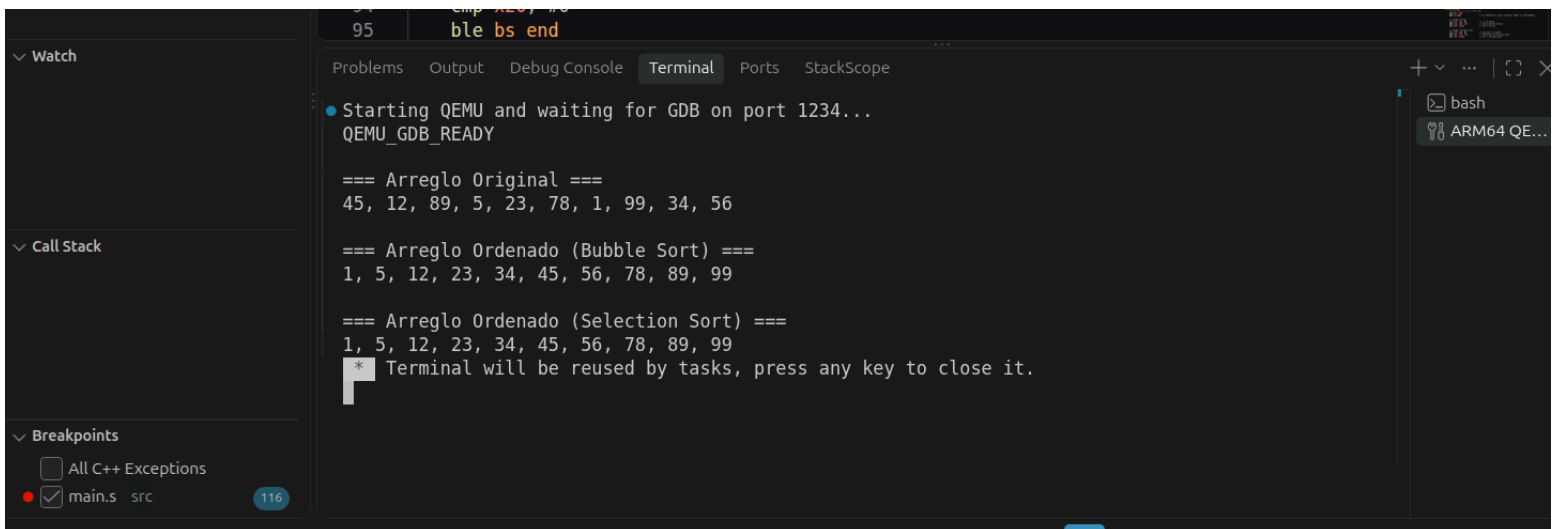
**Carnet:** 202500177

**Curso:** Arquitectura de Computadores y Ensambladores 1 B

**1. Descripción del Proyecto** El presente documento detalla la implementación de los algoritmos de ordenamiento Bubble Sort y Selection Sort desarrollados en lenguaje ensamblador ARM64 puro. El programa procesa un arreglo de 10 enteros de 64 bits inicializado en la sección `.data`, ordenándolo in-place mediante la modificación directa de la memoria.

Se implementó un diseño modular mediante subrutinas independientes, gestionando parámetros a través de los registros `x0` y `x1`, y preservando estrictamente el estado del procesador mediante el resguardo de registros callee-saved en el Stack (`stp x29, x30... / ldp...`).

**2. Evidencia de Ejecución en Consola** A continuación, se presenta la salida de la terminal demostrando el arreglo en su estado original y su estado modificado exitosamente tras la ejecución de ambos algoritmos modulares.



```
95      ble bs_end

Starting QEMU and waiting for GDB on port 1234...
QEMU_GDB_READY

=== Arreglo Original ===
45, 12, 89, 5, 23, 78, 1, 99, 34, 56

=== Arreglo Ordenado (Bubble Sort) ===
1, 5, 12, 23, 34, 45, 56, 78, 89, 99

=== Arreglo Ordenado (Selection Sort) ===
1, 5, 12, 23, 34, 45, 56, 78, 89, 99
* Terminal will be reused by tasks, press any key to close it.
```

**3. Evidencia de Depuración con GDB** La siguiente captura demuestra una sesión de depuración activa utilizando GDB integrado en VS Code a través del emulador QEMU. Se estableció un breakpoint dentro del ciclo interno del algoritmo de ordenamiento, permitiendo observar la inspección en tiempo real de los registros del procesador durante la fase de comparación condicional de elementos adyacentes de la memoria.

